



The Lakehouse That Finally Embraces the Database

Graziano Montanaro

PyData Amsterdam | 11 September 2026



Lakehouse Metadata Got Complicated

Manifests all the way down

Iceberg and Delta add metadata layers that also need compaction.

Extra catalog infrastructure

Production Iceberg often adds Hive, Glue, Nessie, or a REST catalog.

Object stores are high-latency metadata stores

Many small metadata reads add latency and request cost on object storage.

Compute-heavy maintenance

Compaction may need a separate Spark job—even for modest tables.

What if table metadata lived in a database?

What is DuckLake?



SQL Catalog

Metadata lives in PostgreSQL, SQLite, or DuckDB—not manifest files.



Parquet Data Files

Immutable Parquet stays on object storage or local disk.



Real ACID Transactions

Catalog commits provide multi-table ACID and snapshot isolation.



Time Travel & CDC

Query any snapshot or read changes between versions.

How DuckLake Works



Key Insight

- ✓ Metadata lookups are SQL queries against an indexed DB, not file-listing operations on S3
- ✓ Data files are immutable Parquet: no lock contention, trivial to cache and replicate
- ✓ Multiple clients can share PostgreSQL; SQLite uses lock retries for local multi-process access

SETUP EXAMPLE

Production Setup: PostgreSQL + S3

SQL

```
INSTALL ducklake; INSTALL postgres;
LOAD ducklake;    LOAD postgres;
-- Attach: PostgreSQL catalog + S3 data path
ATTACH 'ducklake:postgres:host=catalog.prod.internal
       dbname=ducklake_catalog user=lake_admin password=****'
AS prod_lake (DATA_PATH 's3://acme-data-lake/warehouse/');
USE prod_lake;
-- Ingest raw Parquet into a managed DuckLake table
CREATE TABLE orders AS
SELECT * FROM read_parquet('s3://acme-raw/orders/*.parquet');
```

Pick the Catalog for the Workload

SQLite	DuckDB	PostgreSQL
<i>Local multi-process and edge</i>	<i>Single-user development</i>	<i>Production and remote writers</i>
Strengths + Data inlining + Multi-process, no server + Lightweight deployment	Strengths + Fast in-process catalog + Data inlining + One portable file	Strengths + MVCC concurrency + Mature HA and backups + Remote clients
Weaknesses - Writer lock contention - No remote access - Retries under load	Weaknesses - One client - No remote access - Not for multi-user production	Weaknesses - No VARIANT inlining - Another service to run



Avoid MySQL for now: the documented connector has known issues.

Data Inlining: The Small Files Killer

How It Works

- ✓ Small writes stay in the catalog instead of creating tiny Parquet files
- ✓ Queries merge catalog rows and Parquet transparently
- ✓ Flush one table explicitly; CHECKPOINT runs full maintenance
- ✓ Streaming ingestion without thousands of tiny files

⚠ Default in v1.0: 10-row threshold. DuckDB, PostgreSQL, and SQLite catalogs are supported; VARIANT inlining requires a DuckDB catalog.

Data Inlining: The Small Files Killer

How It Works

- ✓ Small writes stay in the catalog instead of creating tiny Parquet files
- ✓ Queries merge catalog rows and Parquet transparently
- ✓ Flush one table explicitly; CHECKPOINT runs full maintenance
- ✓ Streaming ingestion without thousands of tiny files

Benchmark: 1K Streaming Inserts

923x

faster queries vs. Iceberg

105x

faster ingestion vs. Iceberg

Iceberg writes ~4 S3 files per batch
(data + manifest + manifest-list + metadata)

Source: [ducklake.select/2026/04/02/data-inlining-in-ducklake](#)

⚠ Default in v1.0: 10-row threshold. DuckDB, PostgreSQL, and SQLite catalogs are supported; VARIANT inlining requires a DuckDB catalog.

COMPARISON

DuckLake vs Iceberg vs Delta Lake

	DuckLake	Apache Iceberg	Delta Lake
Metadata	SQL database	Avro manifests	JSON log
Catalog	Built in	External / optional server	Optional server
Small writes	Data inlining	Files + metadata	Files + log
Transactions	Multi-table ACID	Table-level	Table-level
Engines	DuckDB; emerging clients	Broad ecosystem	Broad ecosystem
Concurrency	Catalog-backed OCC	Mature OCC	Log-based OCC
Governance	Catalog + storage IAM	Mature catalogs	Unity ecosystem
Track record	New; production-ready spec	Petabyte proven	Petabyte proven
Operations	Low–medium	High	Medium

DuckLake trades ecosystem maturity for multi-table ACID and a simpler metadata path.

v1.0 Shipped. What's Next?

SHIPPED IN v1.0



Data Inlining (Insert / Delete / Update)

Default-on small-write inlining across DuckDB, PostgreSQL, and SQLite catalogs.



Multi-Engine Clients & New Types

Spark, DataFusion, Trino, and Pandas clients; GEOMETRY, VARIANT, and bucket partitions.



Sorted Tables + Deletion Vectors

Sorted writes plus experimental Puffin deletion vectors.

STILL ON THE ROADMAP



v1.1 (Fall 2026, tentative): VARIANT Inlining

Non-DuckDB catalog support; multi-deletion-vector Puffin files.



Future: Git-like Branching

Branch and merge table histories; no committed release date.



Future: Roles, ACLs, Incremental Mat. Views

Native permissions and change-feed-backed materialized views.

v1.0 is production-ready. Roadmap items and dates are tentative: ducklake.select/roadmap

BETA

Quack: DuckDB Goes Client-Server

What Is Quack?

- ✓ DuckDB-to-DuckDB over HTTP
- ✓ Concurrent remote writers
- ✓ One-round-trip queries
- ✓ 60M rows transferred in under 5s
- ✓ Token auth; pluggable callbacks

What It Unlocks for DuckLake

DuckDB as Catalog Server

A remote DuckDB instance can serve the catalog. Still beta.

Inlining + Concurrency

DuckLake catalog support ships in v1.5.3+ and remains beta.

Wasm Support

Browser DuckDB can connect directly to Quack.

Timeline

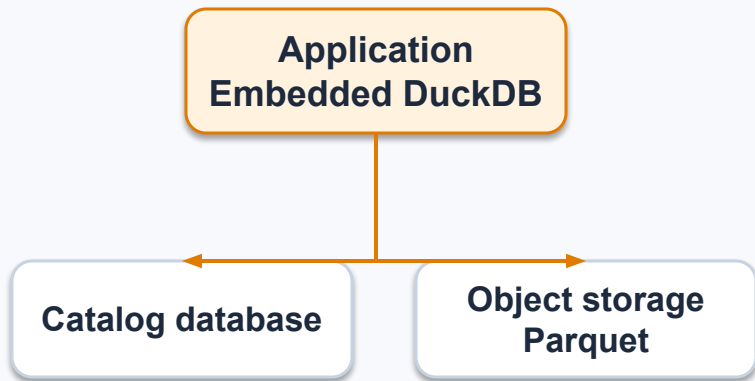
- **Released May 12, 2026**
Quack beta ships as a core extension in DuckDB v1.5.3+

- **Tentative roadmap: Fall 2026**
Target: Quack GA with DuckDB v2.0

- **DuckLake catalog support**
Available in DuckDB v1.5.3+ (beta)

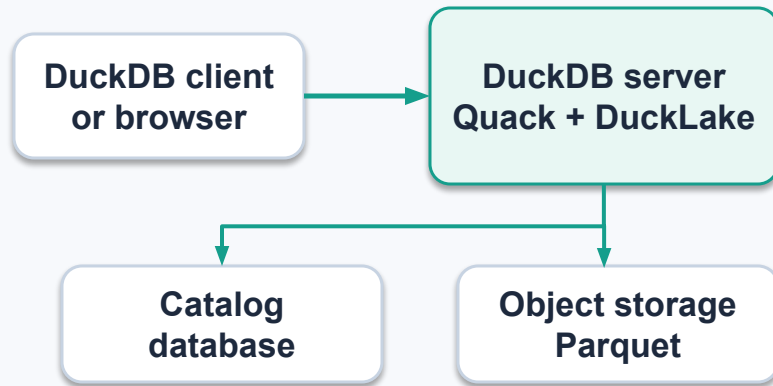
Quack Changes the Access Path

DIRECT DUCKLAKE ACCESS



Each client needs both sets of credentials.

ACCESS THROUGH QUACK



Clients use one endpoint and one access token.

Where DuckLake Still Needs Work



"Isn't this just Hive Metastore?"

Both use a database, but DuckLake is the table format: ACID, snapshots, schema evolution, and time travel.



VARIANT inlining: DuckDB-catalog only

With PostgreSQL or SQLite catalogs, VARIANT rows bypass inlining; other eligible writes still inline.



Catalog DB is now critical infrastructure

The catalog needs HA, backups, and monitoring. Heavy writer concurrency can exhaust retries.



Governance and tooling are not there yet

Native roles and lineage are not built in; access control comes from the catalog and object store.

Three Stacks, Three Trade-offs

PySpark + Iceberg

Strengths

- + Proven at petabyte scale
- + Broad engine support
- + Mature governance
- + Deep ecosystem

Weaknesses

- Heavy JVM/K8s stack
- Separate compaction jobs
- Overkill for smaller data

Polars + Delta Lake

Strengths

- + Fast single-node engine
- + Python-native; no JVM
- + Battle-tested Delta
- + Growing Rust ecosystem

Weaknesses

- Writes depend on delta-rs
- Feature gaps vs Spark
- Single-node ceiling

DuckDB + DuckLake

Strengths

- + Two-command setup
- + Small-write inlining
- + Time travel + CDC
- + Multi-table ACID

Weaknesses

- Young connectors
- No native fine-grained ACLs
- New production track record

Choose for your team, engines, and scale—not the feature checklist.

Should You Actually Use This?

Use DuckLake When...

- ✓ Moderate writer concurrency
- ✓ Small team, minimal ops
- ✓ DuckDB is already your engine
- ✓ Time travel + CDC, no Kafka
- ✓ Local-first, cloud-ready
- ✓ Validate a lakehouse quickly

Don't Use DuckLake When...

- ✗ Mature Flink, Polars, or Ray support
- ✗ Hundreds of writers at petabyte scale
- ✗ Built-in governance and lineage
- ✗ Your Iceberg stack already works
- ✗ Native row/column security
- ✗ A long production track record

DuckLake is a different operational trade-off—not an Iceberg replacement.

Code demo

Three Things to Remember

1

Metadata becomes a database

SQL metadata replaces manifest traversal and many small object-store reads.

2

The stack stays small

DuckDB + DuckLake delivers ACID, time travel, CDC, and schema evolution in a small stack.

3

Fit matters more than novelty

Catalog choice and writer concurrency set the limits; mature multi-engine deployments still favor Iceberg.



The Lakehouse That Finally Embraces the Database

One database for metadata. One bucket for data. That's the whole lakehouse.

GitHub

`github.com/montanarograziano/
ducklake_playground`

Website

`ducklake.select`

Docs

`ducklake.select/docs/stable`



LinkedIn

Questions?

Graziano Montanaro | graziano.montanaro@intella.tech

